



软件工程导论

Li Qicheng (李起成)
College of Computer Science
Nankai University

*Slides adapted from materials by Prof. Qiang Liu (Tsing Hua University),
Xin Zou (BeiHang University) and Ivan Marsic (Rugers University)*

本节内容

- 什么是软件工程
- 什么是软件工程师
- 什么是软件工程的目的是
- 什么是软件质量
- 如何提高软件质量
- 什么是个人软件流程（PSP）

软件工程诞生

1968年，北大西洋公约组织（NATO）召开国际会议，
提出 “软件工程” 概念和术语。



“工程” 的含义



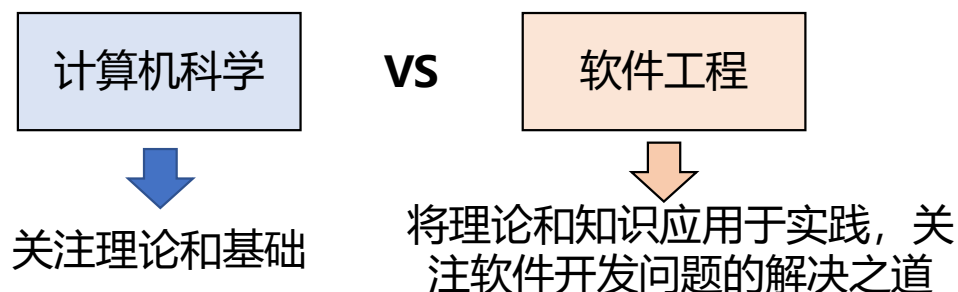
- 大规模的设计与建造
- 复杂问题与目标分解
- 团队协作与过程控制

工程是应用有关的科学知识和技术手段，通过有组织的群体协作活动，建造具有预期使用价值的人造产品的过程

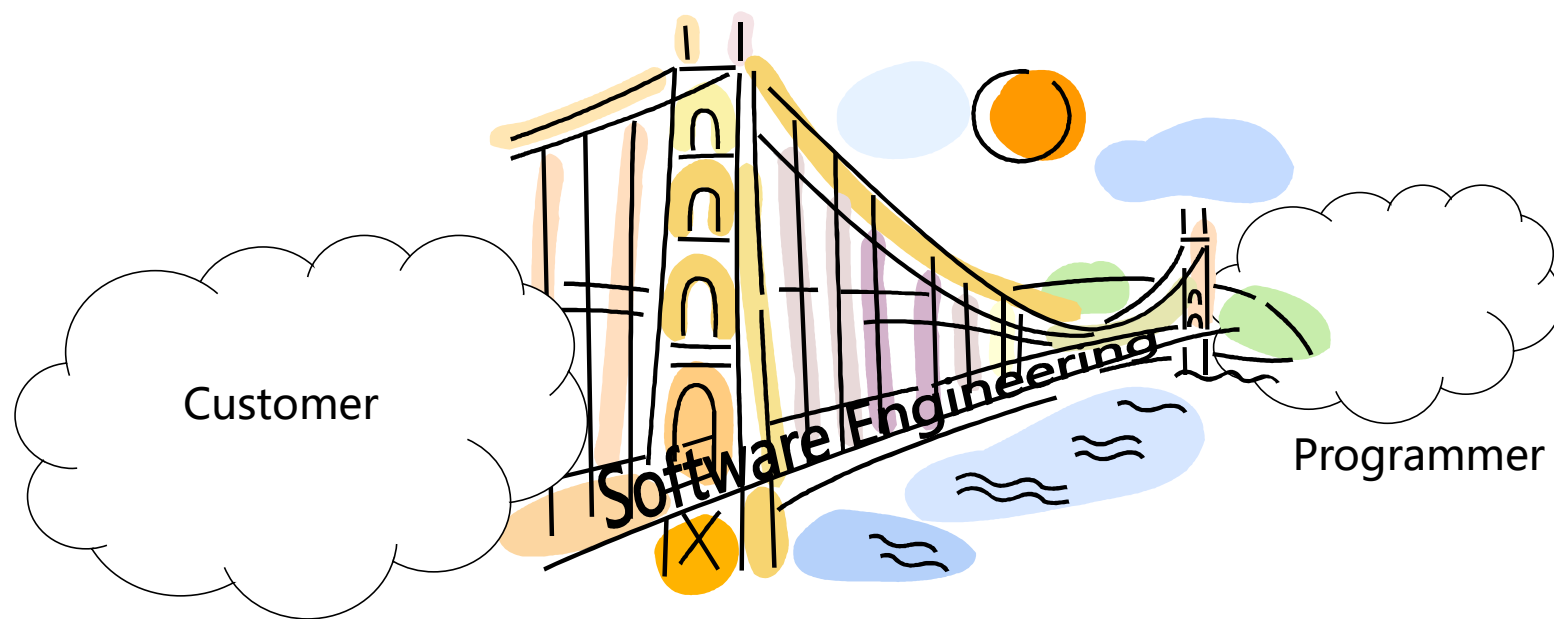


什么是软件工程？

- 软件工程是将系统性的、规范化的、可量化的方法应用于软件的开发、运行和维护上，并对这种方法进行研究。
- 软件工程是一门**工程学科**，涉及软件生产的各个方面，从最初的需求规格说明直到投入使用后的系统维护，都属于其学科范畴。

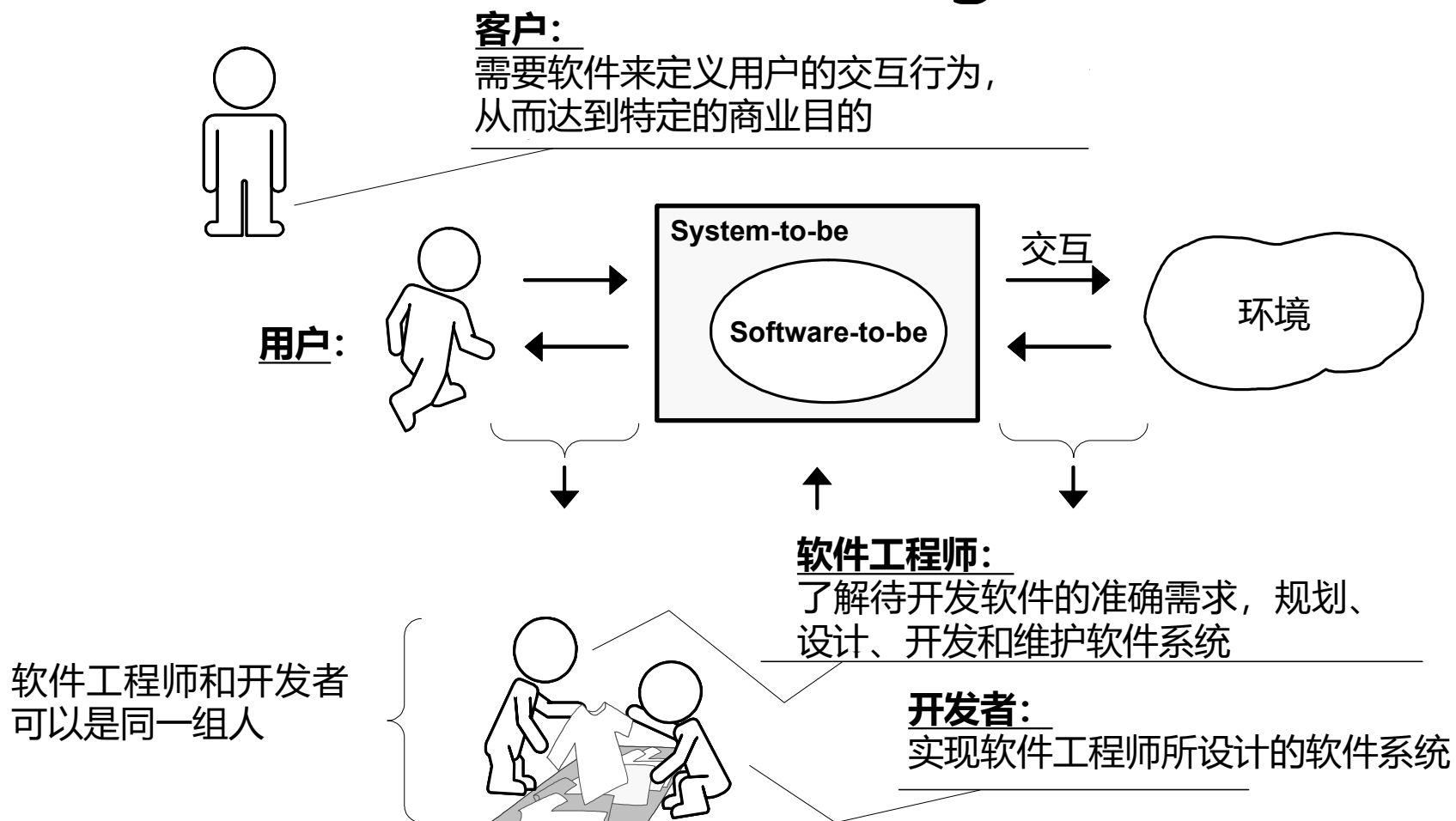


什么是软件工程？



软件工程是从**客户需求**到**编程实现**之间的桥梁。

软件工程师 (Software Engineer)



此题未设置答案，请点击右侧设置按钮

下面的（ ）说法是正确的

提交

- A 由于软件是产品，因此可以应用其他工程制品所用的技术进行生产
- B 购买大多数计算机系统所需的硬件比软件更昂贵
- D 大多数软件系统是不容易修改的，除非它们在设计时考虑了变化
- C 一般来说，软件只有在其行为与开发者的目标一致的情况下才能成功

此题未设置答案，请点击右侧设置按钮

造成大型软件开发困难的根本原因在于（ ）。

提交

- ☐ A 开发人员缺乏足够的开发经验
- ☐ B 对软件开发的资金投入不足
- ☐ C 项目开发进度不合理
- ☐ D 软件系统的复杂性

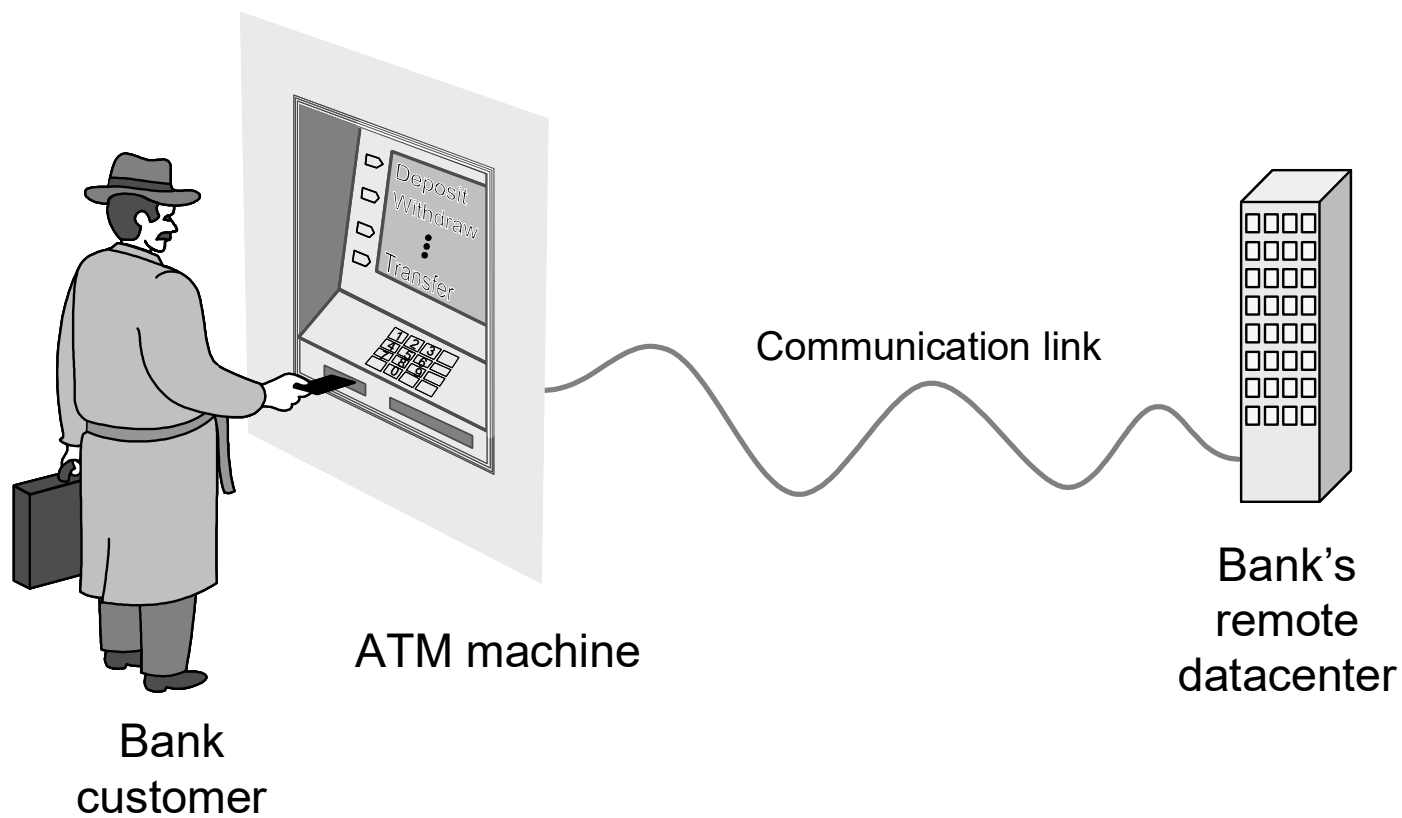
此题未设置答案，请点击右侧设置按钮

软件会逐渐退化而不会磨损，其原因在于（）。

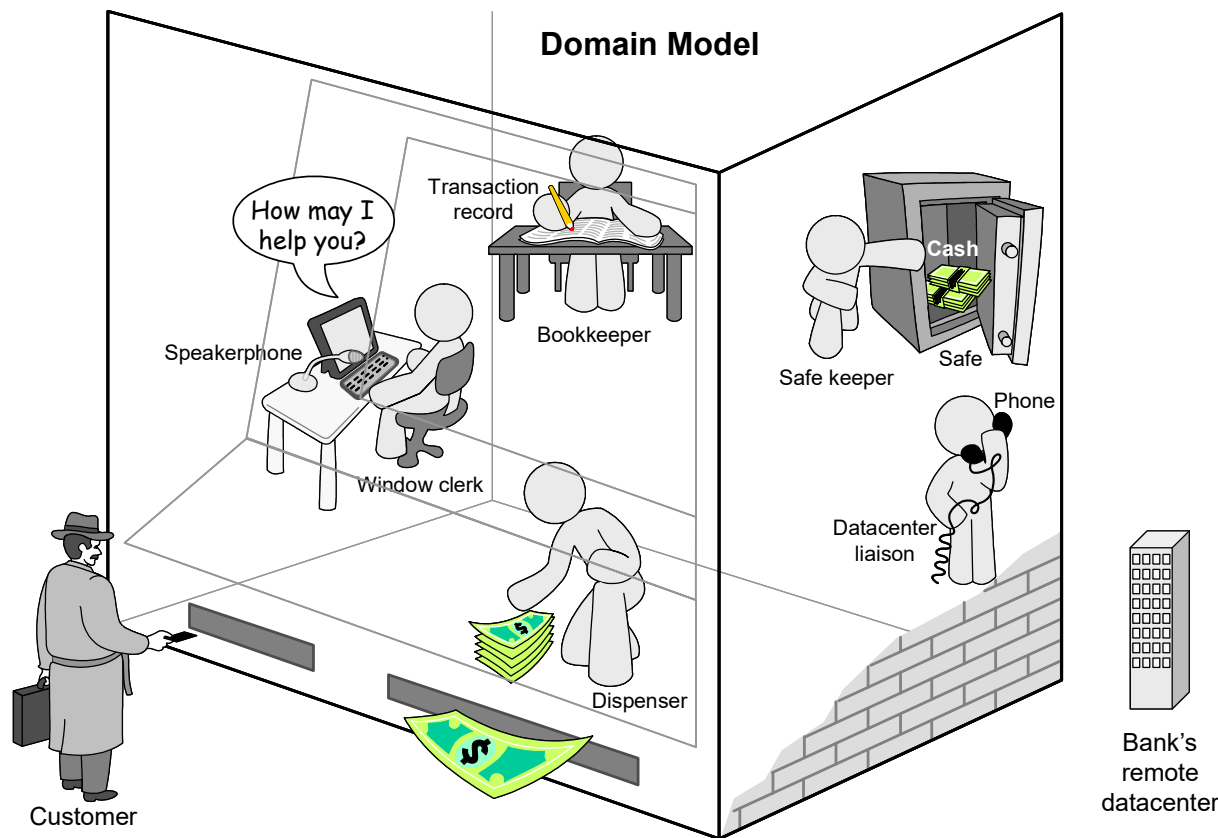
提交

- ☐ A 软件通常暴露在恶劣的环境下
- ☐ B 软件错误在经常使用之后会逐渐增加
- ☐ D 不断的变更使组件接口之间引起错误
- ☐ C 软件备件很难订购

举例：ATM

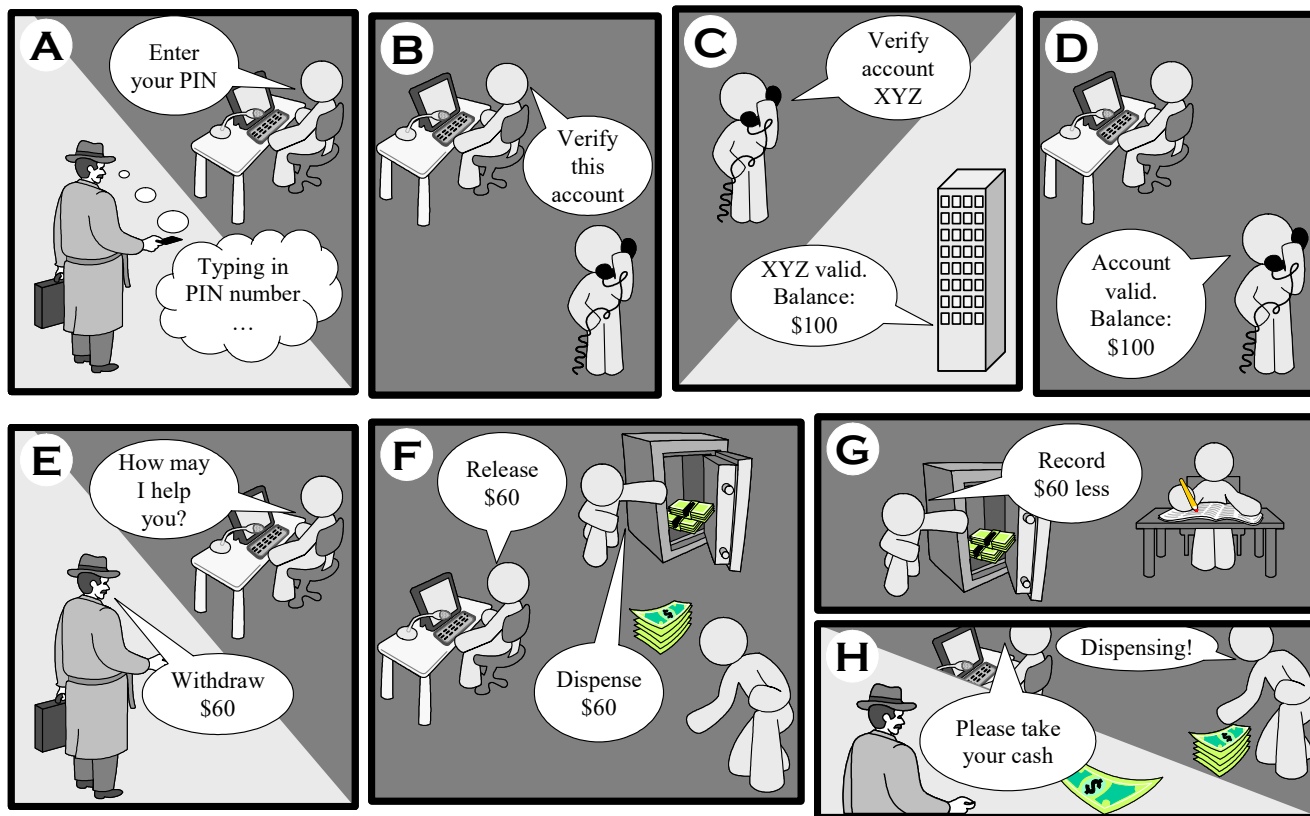


Step1: 定义问题，掌握需求



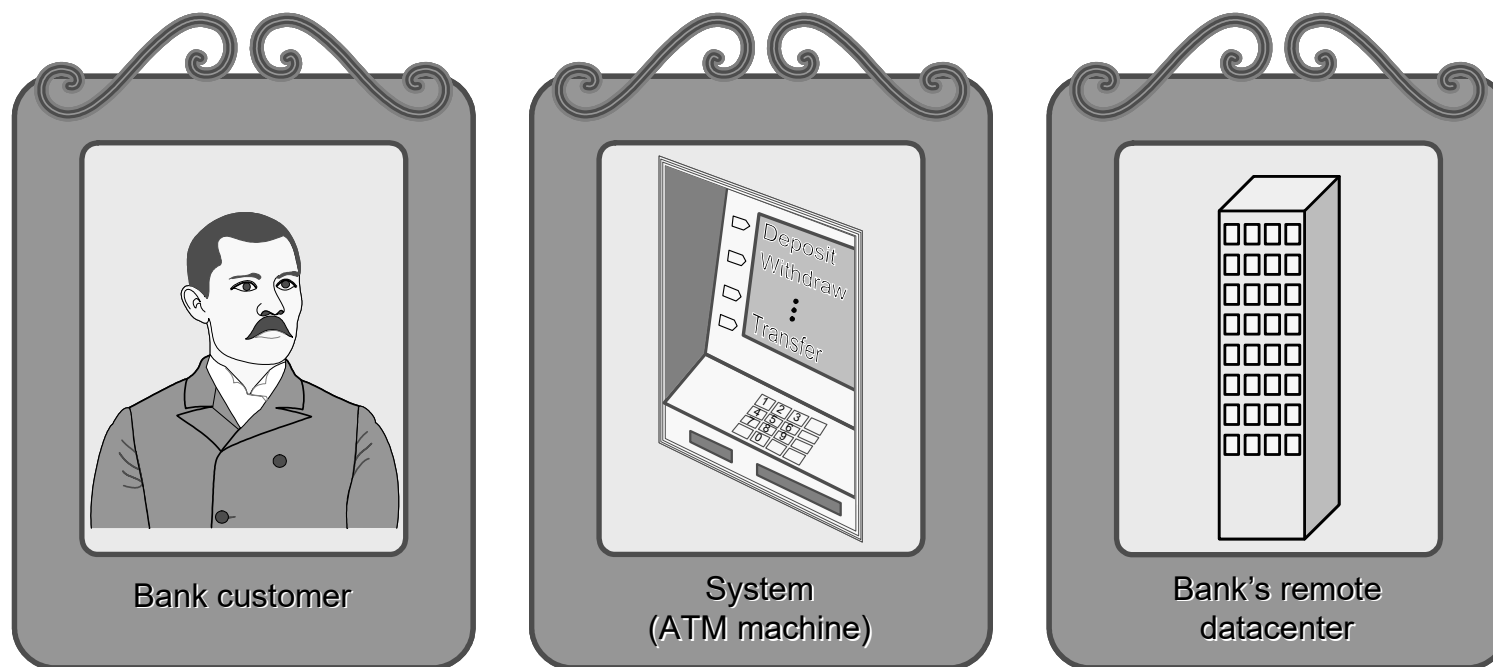
- 领域模型又叫做概念模型，专注于分析问题领域本身，是领域内的概念类或现实世界中对象的可视化表示。
- 在领域专家的帮助下，构建领域模型。
- 分而治之：将整个系统划分为各个逻辑模块，每个模块解决一个单独的子问题。

Step1: 理解问题，掌握需求（二）



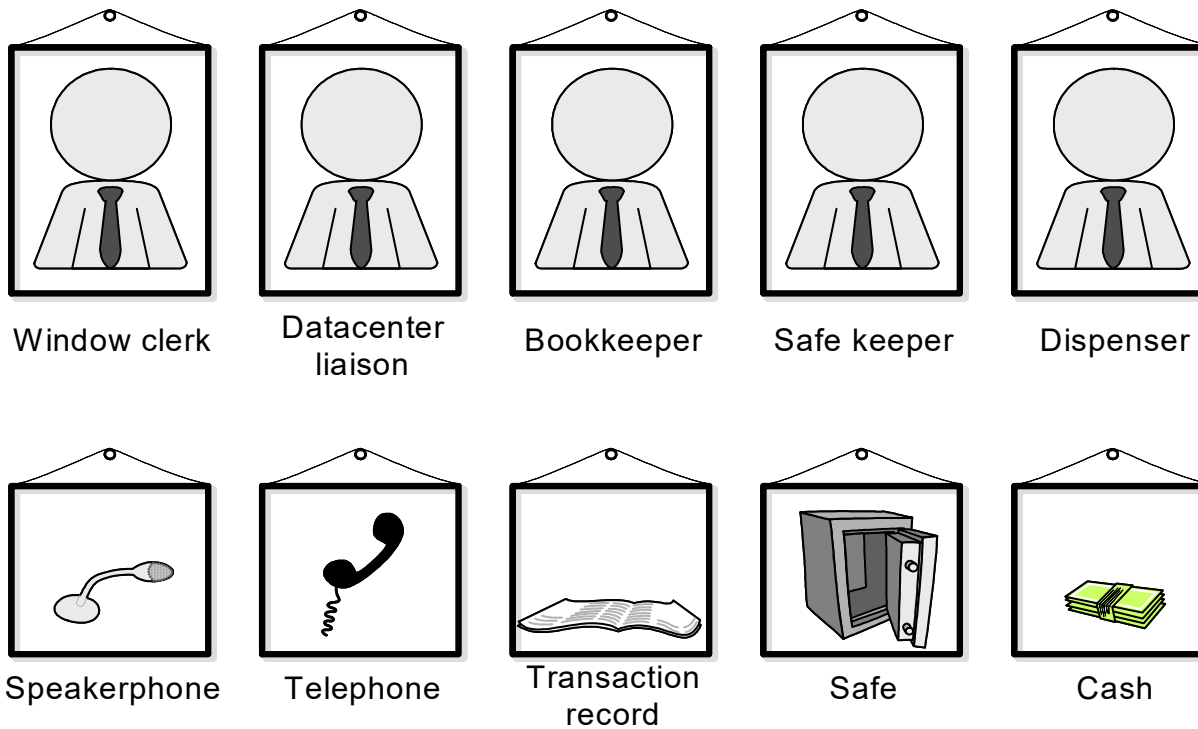
可通过简单手绘的方式，描述待开发软件的需求。

Step2: 抽取交互对象



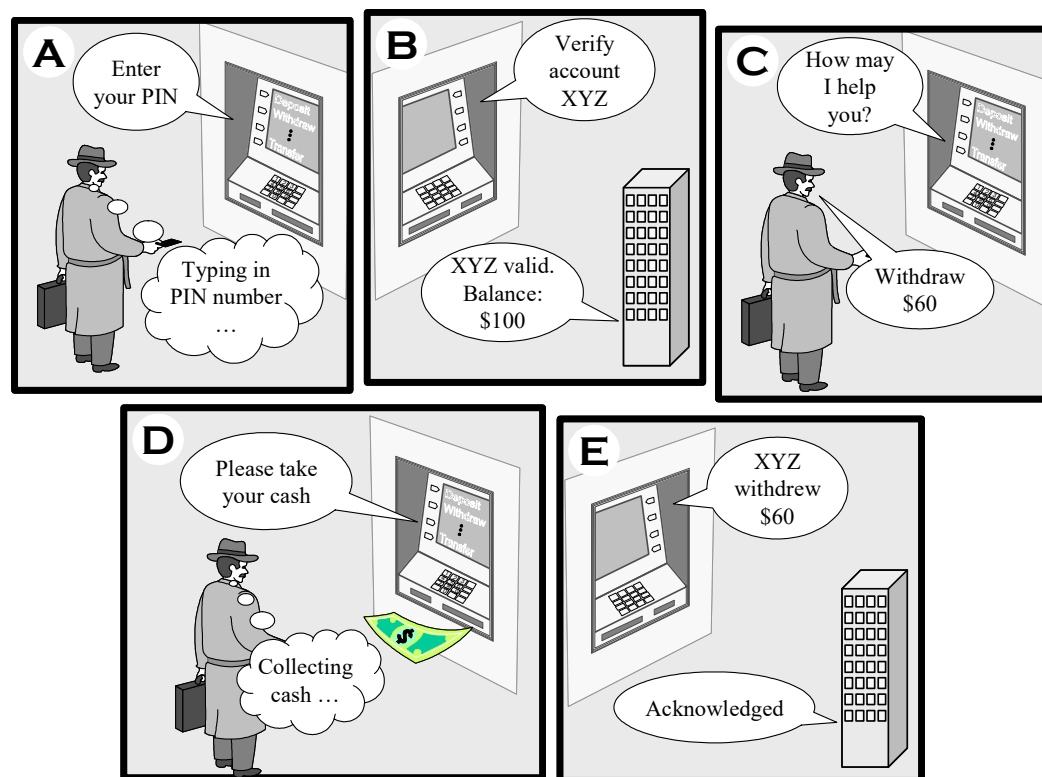
- 交互对象：存在于软件**外部**、与软件进行交互的对象
- 由于其可见性，通常较易识别

Step3: 抽取概念对象



- 概念对象：软件内部，实现软件功能的概念和对象
- 由于其不可见性，通常较难识别。

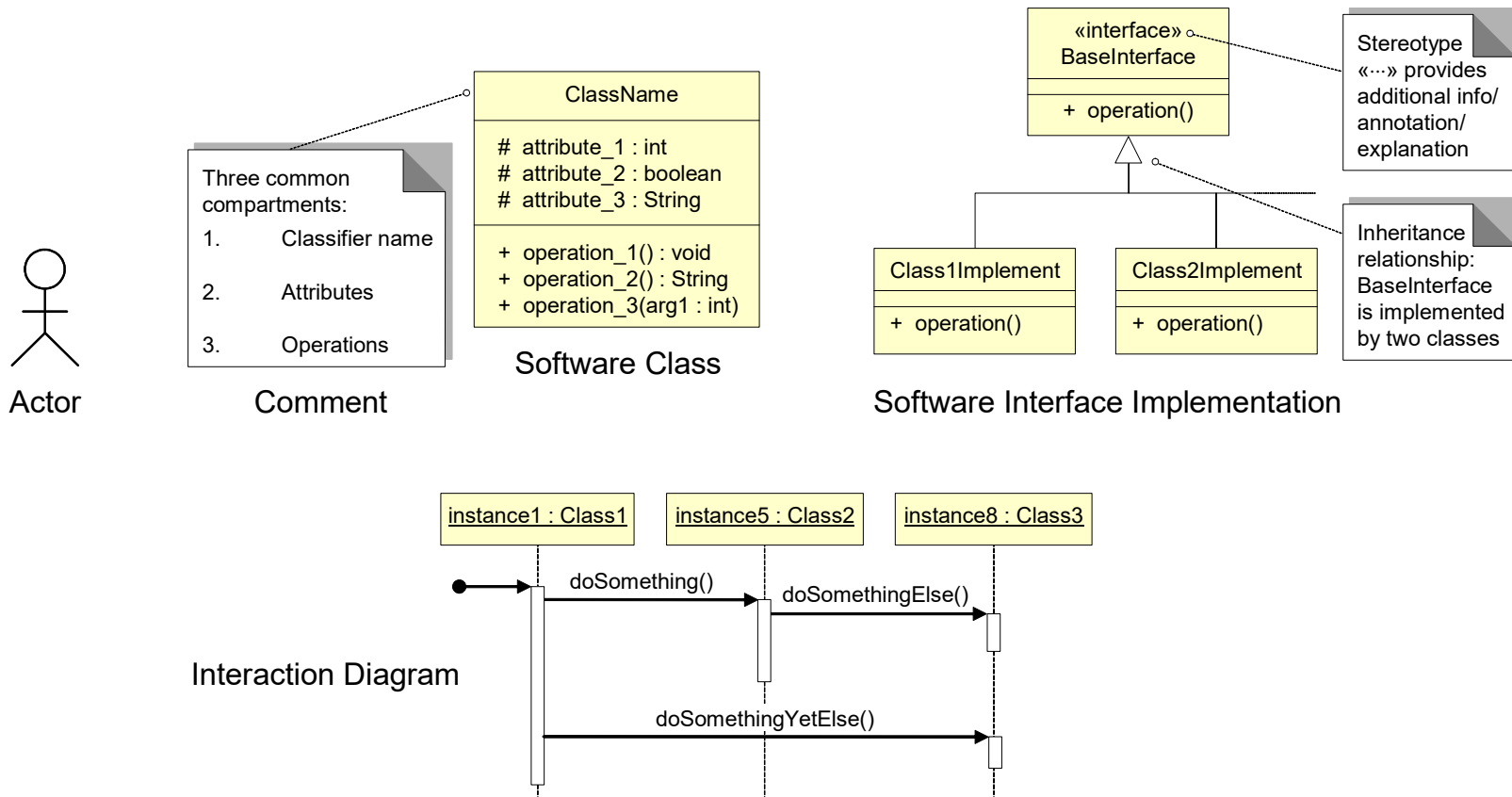
Step4: 抽取用例场景



场景一：取钱

...

Step5: 系统建模



Step5: 项目管理

把控项目开发过程中的三个要点:

- 预算
- 时间
- 质量

Size(①) = 4

Size(②) = 7

Size(③) = 10

Size(④) = 3

估算项目开发时间:

Size(⑤) = 4

- 估算整个开发中某个小部分的工作量

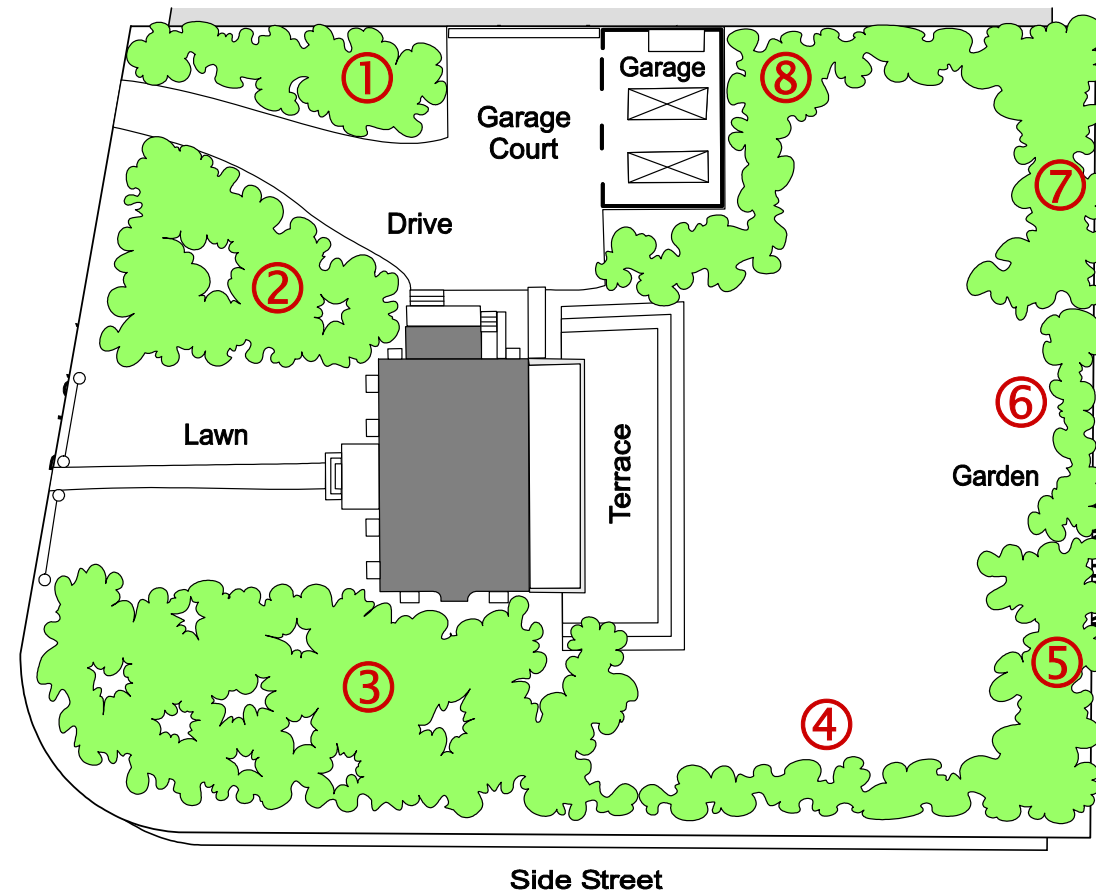
Size(⑥) = 2

- 提高团队开发效率, 完成该部分开发工作

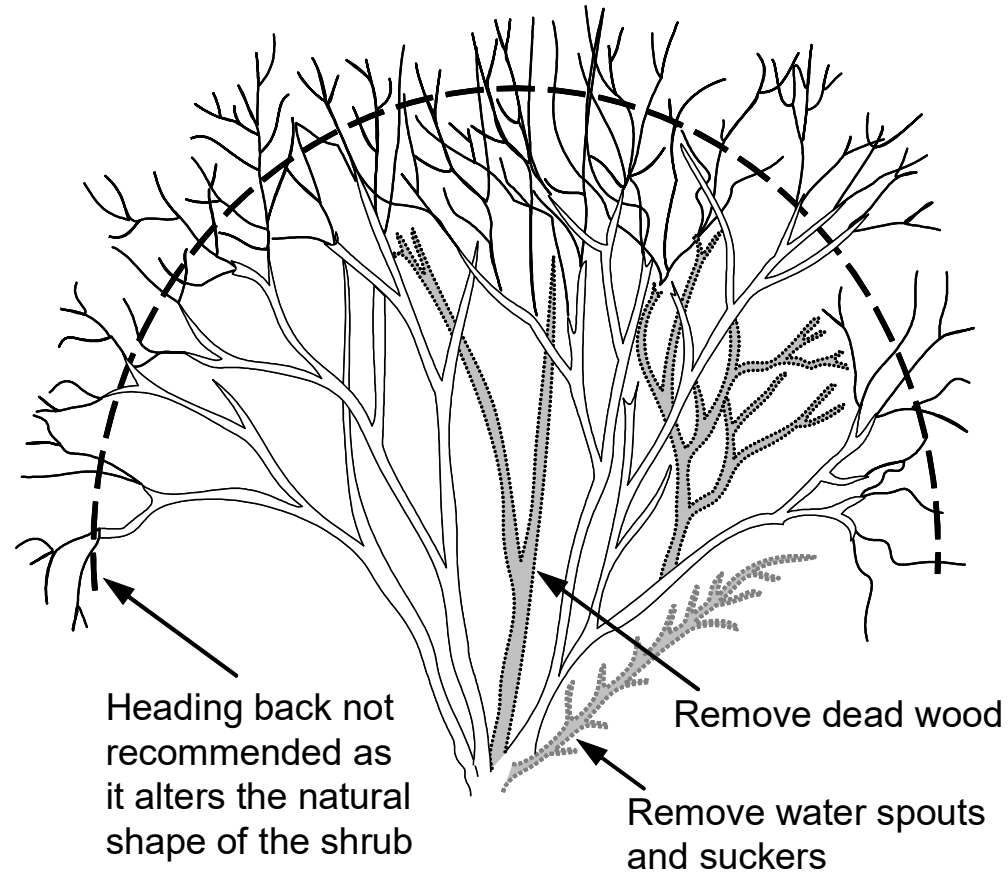
- 根据实际时间不断调整项目的整体估测时间

- 重复上述过程, 直至无需再修改或项目完成

Size(⑧) = 7

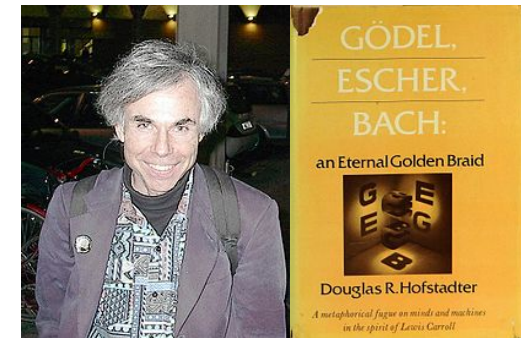


Step6: 软件质量评估



软件工程的规律

- 时间估计霍夫斯塔特定律：
 - **实际时间总是比预期要长，即便你考虑到了霍夫斯塔特定律。**
 - 这个规律如何应用到实际中呢？
- 增加人手：
 - **向进度落后的项目中增加人员，会让项目更加落后。**
 - 有数量的关系么？
 - 如果是增加专家呢？
 - 搞 996 呢？



“客户”与“用户”的区别是什么？

作答

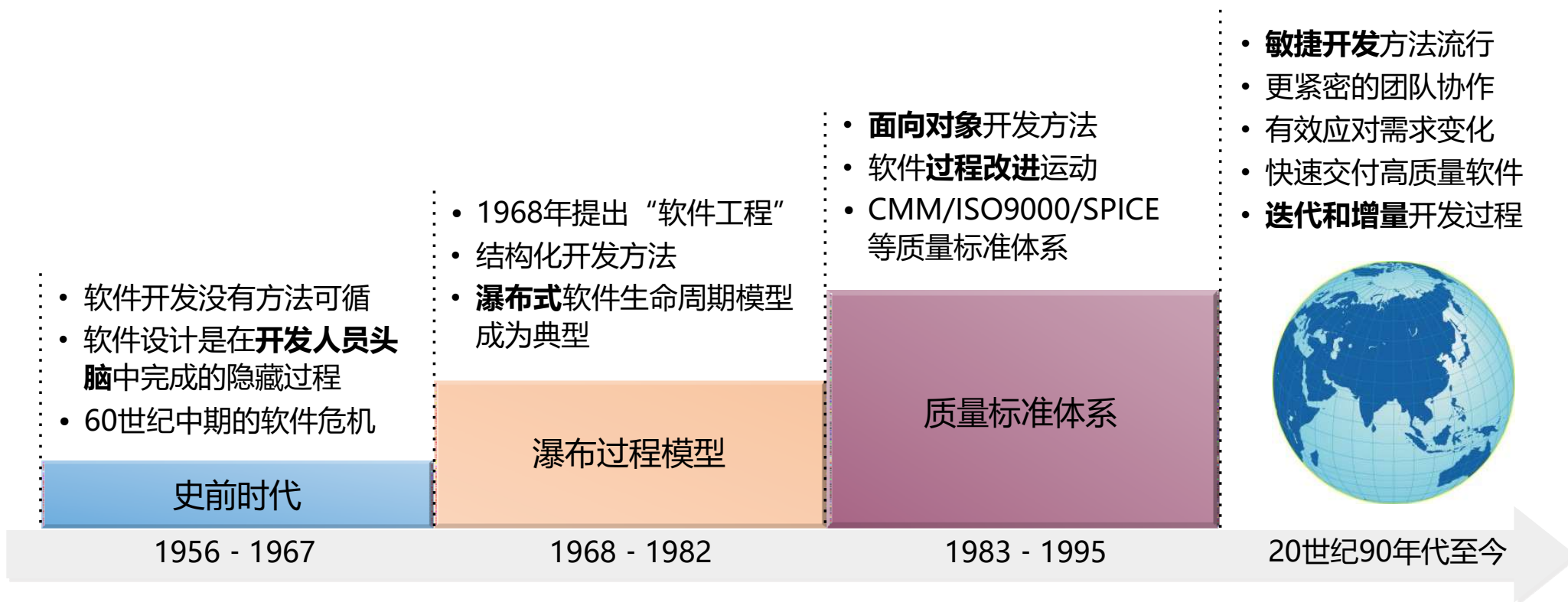
软件工程师与软件开发人员的有什么区别？可为同一人吗？

作答

软件开发各个环节：定义问题、掌握需求、抽取交互对象、抽取概念对象、系统建模、编程实现、项目管理、质量评估、测试等等，一定是以线性顺序进行的吗？

作答

软件工程的发展历史



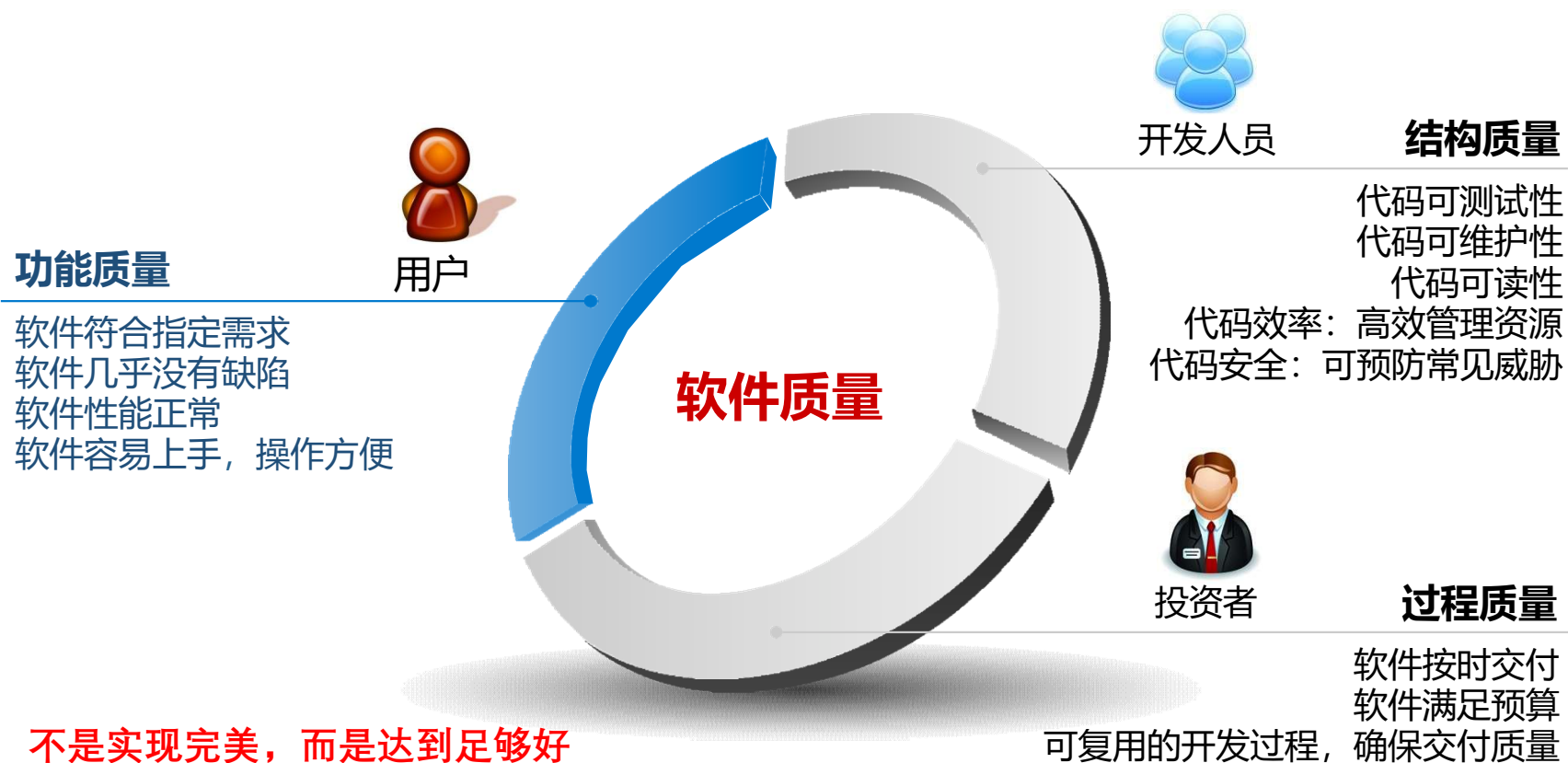
软件工程的主要目的

软件工程的目的是：**经济有效**地开发具有**高质量**的软件系统。

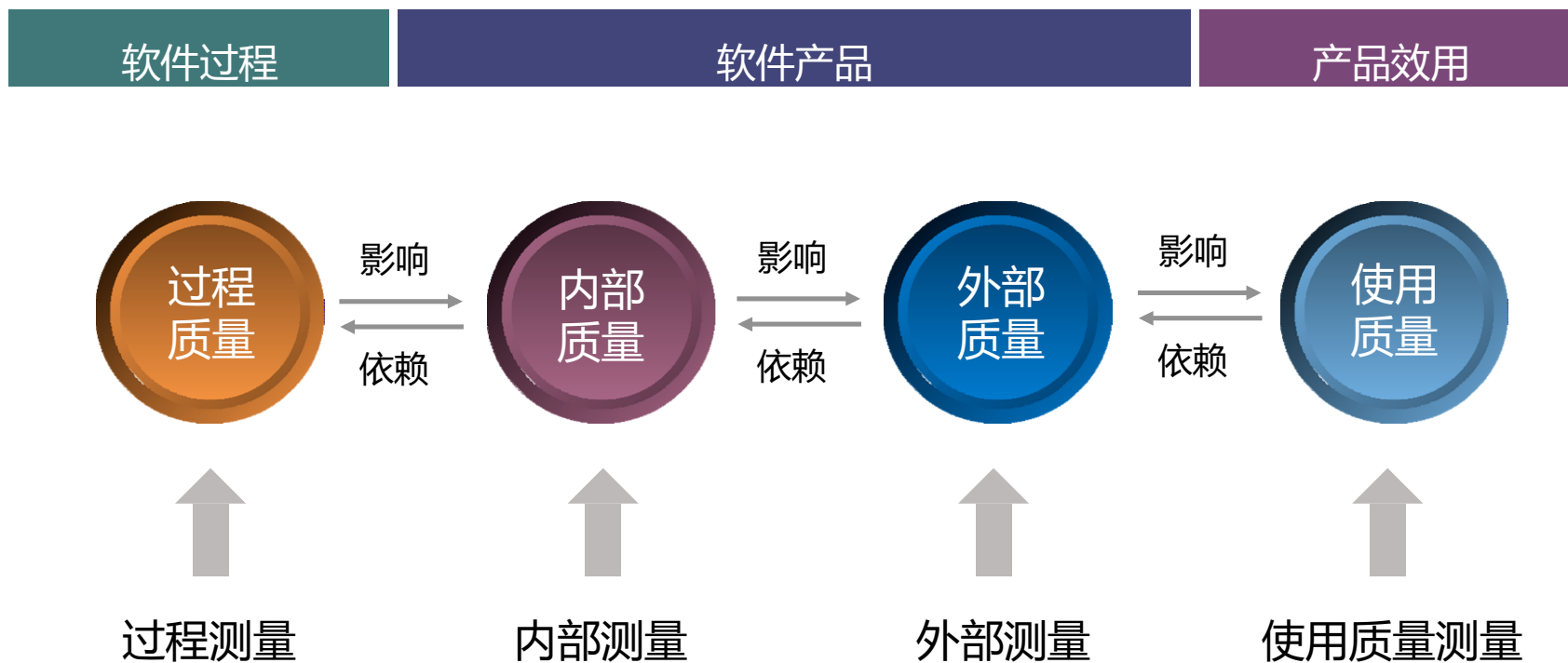
什么是高质量的软件？

- 开发成本较低 (**预算**)
- 按时完成开发任务并及时交付 (**时间**)
- 实现客户要求的功能 (**需求**)
- 具备良好的性能、可靠性、可扩展性、可移植性等 (**高质量**)
- 软件维护成本低 (**可维护**)

什么是好的软件



什么是好的软件



什么是好的软件

质量就是软件产品对于某个（或某些）人的价值。

—— 杰拉尔德·温伯格



正确的
软件



一个软件要能够满足用户的需求，为用户创造价值。这里的价值可以体现在两个方面，即为用户创造利润和减少成本。

软件运行
正确



软件没有或者有很少缺陷，具有很强的扩展性、良好的性能以及较高的易用性等。

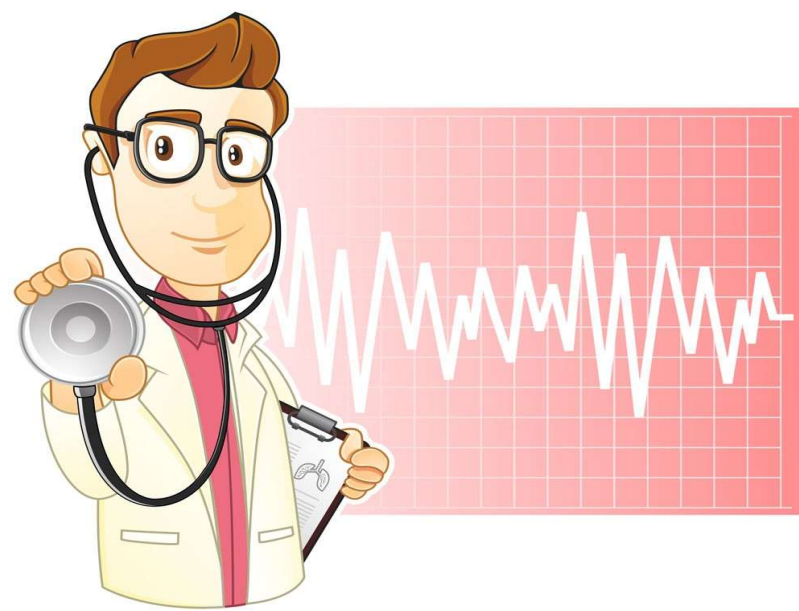
什么是好的软件

高质量的软件产品：

- 做了用户想要它做的事情
- 正确有效地使用计算机资源
- 易于用户学习和使用
- 设计良好、代码规范且易于测试和维护



产品质量维度



健康体检档案

姓名_____性别_____年龄_____

婚姻状况_____联系电话_____

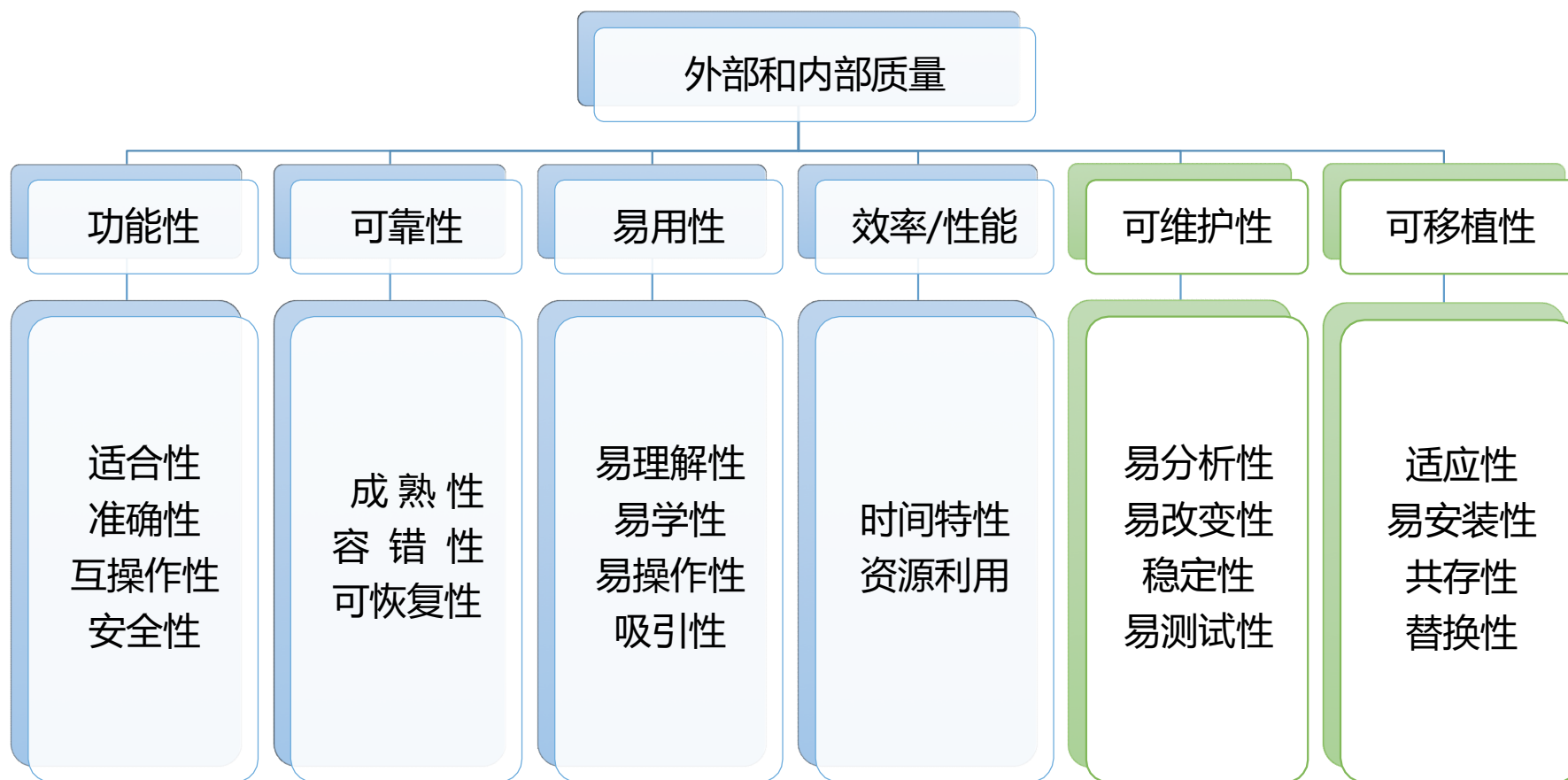
家庭住址：_____

眼科	眼科	辨色力	正常		
		裂隙灯检查右眼	未见异常		
		裂隙灯检查左眼	未见异常		
		其他	屈光不正		
彩超	彩超(肝胆)	肝	肝脏大小形态正常，表面光滑，内部回声分布均匀，肝内胆管显示清晰，血管走向正常，门静脉不宽。		
		胆	胆囊形态大小正常，壁光滑，透声清晰。胆总管不增宽。		
	彩超(双肾)	双肾	双肾大小形态正常，被膜光滑，皮、髓质分界清，集合系统未见分离。		
	彩超(脾)	脾	脾脏大小形态如常，被膜光滑，内部回声均匀，脾门静脉不宽。血流未见异常。		
	彩超(胰)	胰	胰腺大小形态如常，边界清晰，内部回声均匀，主胰管不扩张。		
心电图	12导心电图	12导心电图	窦性心动过缓(心率58次/分)		
放射科	胸部正位片	胸部正位片	两肺野清晰，气管居中，两肺纹理清晰，未见明显活动性病变，两肺门影不大，两肺肋膈角锐利，心脏及大血管未见明显异常。		
年龄		白细胞	4.96	10 ⁹ /L	3.97-10.00
		粒细胞比值	54.3	%	40.0-68.0
		淋巴细胞比值	37.2	%	20.0-40.0
		单核细胞比值	5.8	%	3.0-9.0
		嗜酸粒细胞比值	2.5	%	0.5-8.0
		嗜碱粒细胞比值	0.2	%	0.0-1.2
		粒细胞绝对值	2.69	10 ⁹ /L	2.00-6.8
		淋巴细胞绝对值	1.85	10 ⁹ /L	0.80-4.00
		单核细胞绝对值	0.29	10 ⁹ /L	0.12-0.80
		嗜酸粒细胞绝对值	0.12	10 ⁹ /L	0.02-0.8
		嗜碱粒细胞绝对值	0.01	10 ⁹ /L	0.00-0.10
		血小板	241	10 ⁹ /L	100-300
		血小板平均体积	8.4	fL	7.6-13.2
		血小板分布宽度	16.0		7.0-19.0
	大血小板比率	0.202		0.080-0.430	
	红细胞	4.92	10 ¹² /L	3.50-5.50	
	红细胞压积	43.4	%	38.0-50.8	
	血红蛋白	150	g/L	120-170	
	红细胞分布宽度CV	13.1		11.0-16.0	
	红细胞分布宽度SD	44.8		35.0-56.0	
	红细胞平均体积	88.2	fL	80.0-100.0	
	平均血红蛋白含量	30.5	pg	27.0-34.0	
	平均血红蛋白浓度	346	g/L	310-378	
		比重	1.015		1.005-1.03
		浊度	7.0		5.0-8.0
		亚硝酸盐	阴性		阴性
		尿蛋白	阴性		阴性
		葡萄糖	阴性		阴性
		潜血	阴性		阴性
		尿酸红素	阴性		阴性
	尿酸原	阴性		阴性	

产品质量维度



ISO9126 质量模型



ISO9126 质量模型

功能性

- 适合性：当软件在指定条件下使用，其满足明确和隐含要求功能的能力。
- 准确性：软件提供给用户功能的精确度达到要求。
- 互操作性：软件与其它系统进行交互的能力。
- 安全性：软件保护信息和数据的安全能力。

可靠性

- 成熟性：软件产品避免因软件中错误发生而导致失效的能力。
- 容错性：软件防止外部接口错误扩散而导致系统失效的能力。
- 可恢复性：系统失效后，重新恢复原有的功能和性能的能力。

ISO9126 质量模型

易用性

- **易理解性**：软件显示的信息要清晰、准确且易懂，使用户能够快速理解软件。
- **易学习性**：软件使用户能学习其应用的能力。
- **易操作性**：软件产品使用户能易于操作和控制它的能力。
- **吸引性**： 软件具有的某些独特的、能让用户眼前一亮的属性。

效率

- **时间特性**：在规定的条件下，软件产品执行其功能时能够提供适当的响应时间和处理时间以及吞吐率的能力。
- **资源利用**：软件系统在完成用户指定的业务请求所消耗的系统资源，诸如CPU占有率、内存占有率、网络带宽占有率等。

ISO9126 质量模型

可维护性

- **易分析性**：软件提供辅助手段帮助开发人员定位缺陷原因并判断出修改之处。
- **易改变性**：软件产品使得指定的修改容易实现的能力。
- **稳定性**：软件产品避免由于软件修改而造成意外结果的能力。
- **易测试性**：软件提供辅助性手段帮助测试人员实现其测试意图。

可移植性

- **适应性**：软件产品无需做任何相应变动就能适应不同运行环境的能力。
- **易安装性**：在平台变化后，成功安装软件的难易程度。
- **共存性**：软件产品在公共环境与其共享资源的其他系统共存的能力。
- **替换性**：软件系统的升级能力，包括在线升级、打补丁升级等。

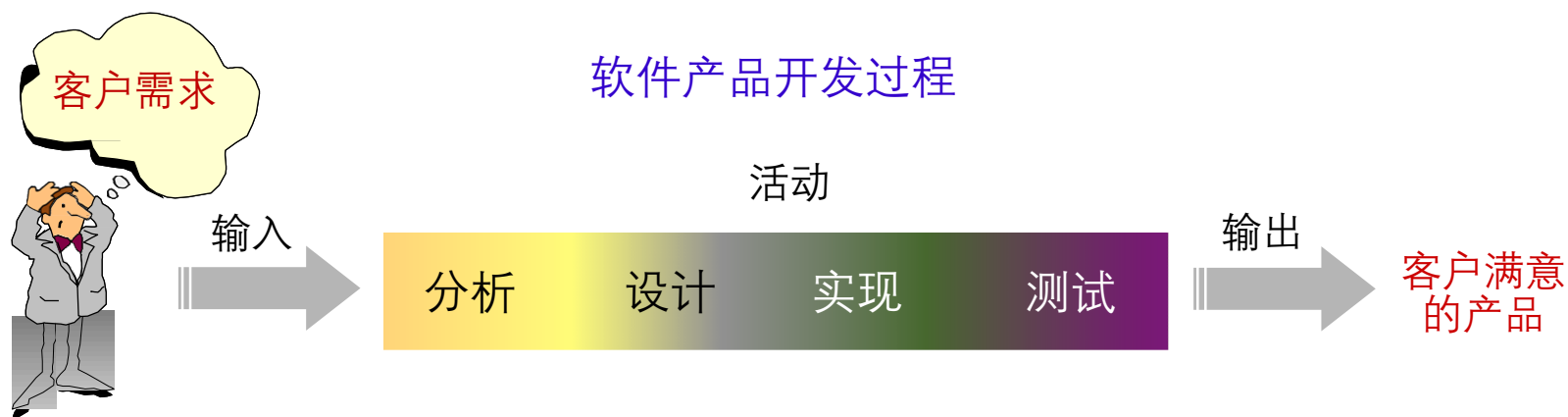
提高软件质量



软件质量是如何实现的？

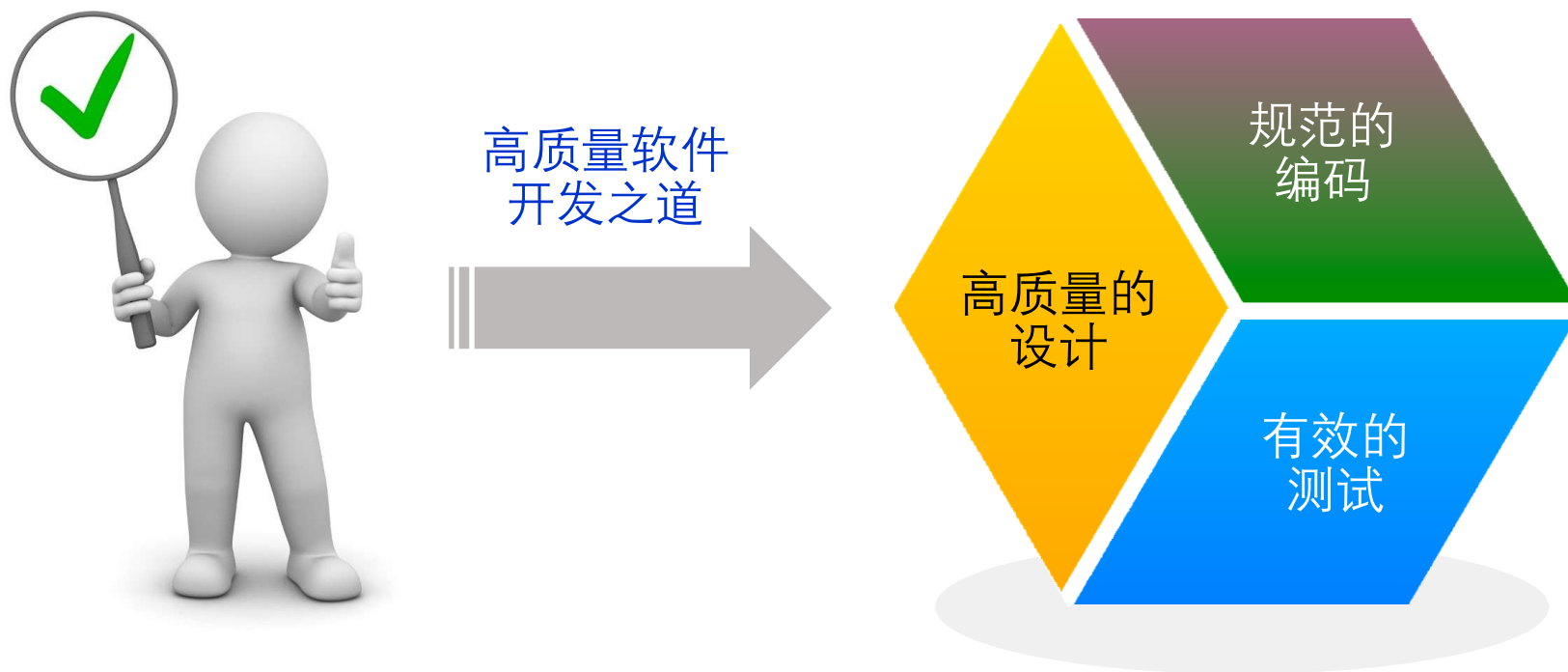
如何才能有效地提高软件质量？

提高软件质量



- 质量不是被测出来的，而是在开发过程中逐渐构建起来的
- 虽然质量不是测出来的，但是未经测试也不可能开发出高质量的软件
- 质量是开发过程的问题，测试是开发过程中不可缺少的重要环节

提高软件质量



思考



软件质量的重要性是无容置疑的，
那么是不是质量越高就越好？

软件产品是否应该追求“零缺陷”？

商业环境下的软件质量



许多互联网软件（例如新浪微博、百度导航等）在产品仍然存在一定缺陷的情况下就发布上线，之后再不断更新版本修复已有的缺陷。



在航天器发射之前，只要发现任何异常，就会立即取消发射指令，直到异常被消除为止。前苏联甚至做得更过分，许多重大武器系统的负责人签了生死状，系统研制成功则获得英雄勋章，失败则被枪毙。

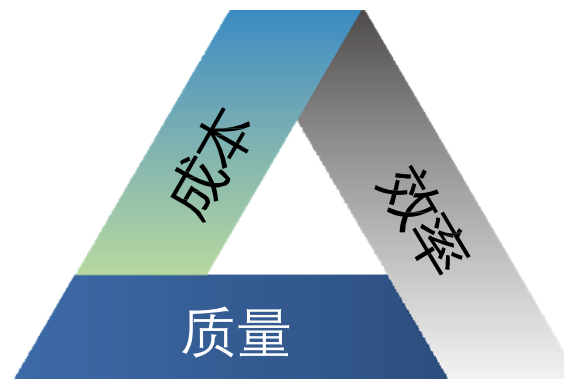
商业软件为什么不像航天系统一样，在发布前应修改所发现的任何缺陷？

商业环境下的软件质量

商业目标决定质量目标：

- 商业目标决定质量目标，不应该把质量目标凌驾于商业目标之上
- 质量是有成本的，不可能为了追求完美的质量而不惜一切代价
- 理想的质量目标不是“零缺陷”，而是恰好让广大用户满意

\$oftware



此题未设置答案，请点击右侧设置按钮

软件工程的基本目标是（ ）。

提交

- ☐ A 开发足够好的软件
- ☐ B 消除软件固有的复杂性
- ☐ C 努力发挥开发人员的创造性潜能
- ☐ D 更好地维护正在使用的软件产品

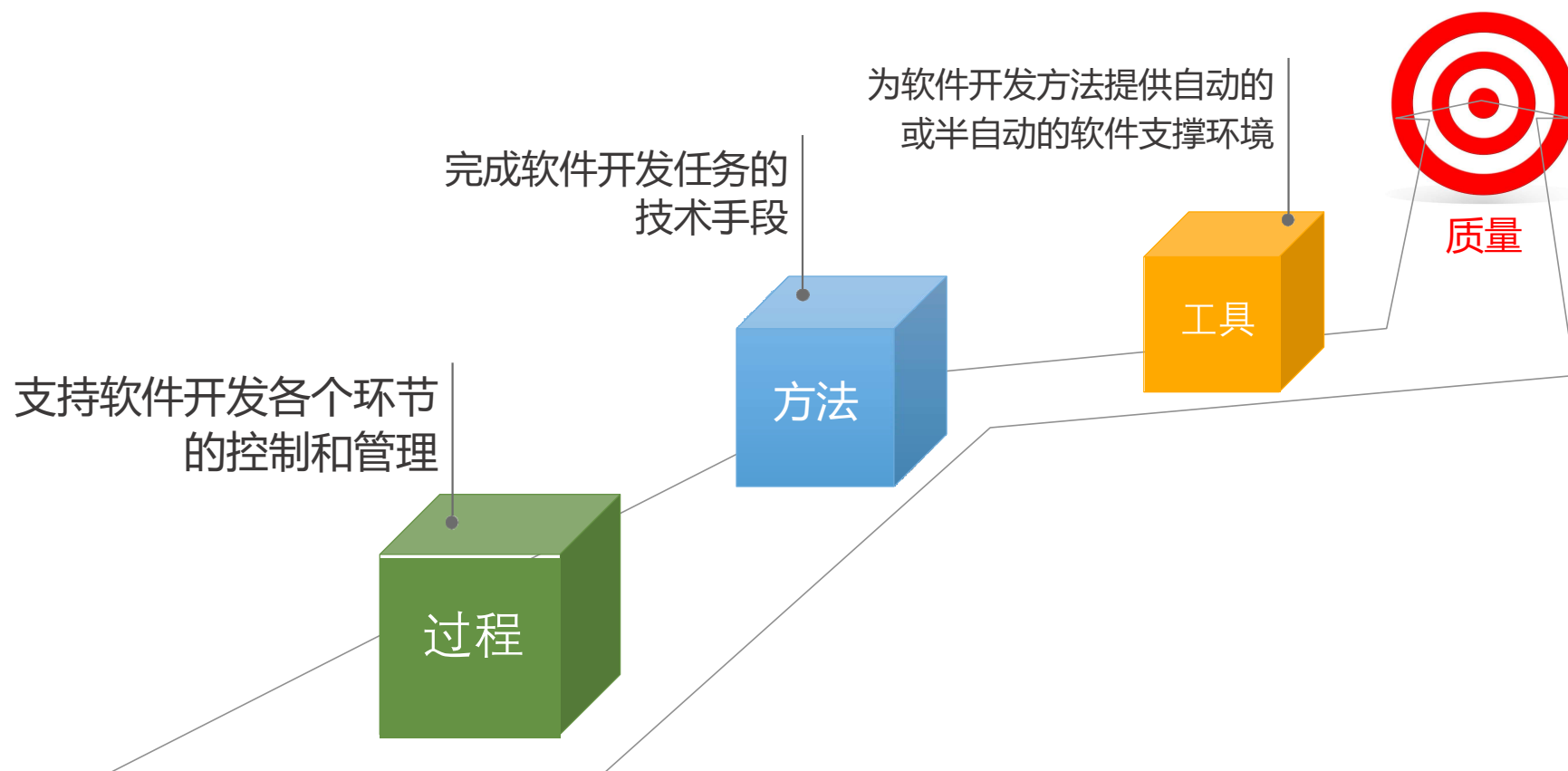
此题未设置答案，请点击右侧设置按钮

下面的（ ）是正确的。

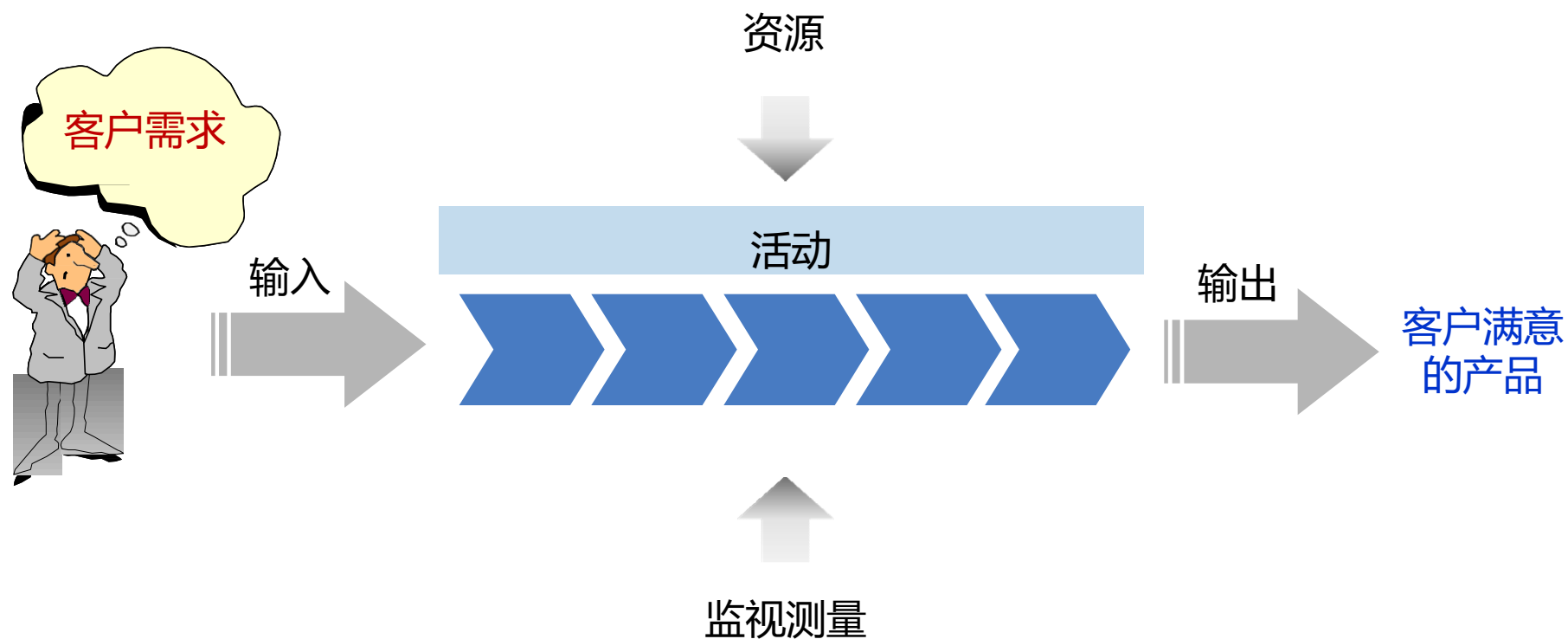
提交

- ☐ A 运行正确的软件就是高质量的软件。
- ☐ B 软件质量是在开发过程中逐渐构建起来的。
- ☐ C 软件产品质量越高越好，最理想的情况是达到“零缺陷”。
- ☐ D 软件质量是由产品的功能、性能、易用性等外在特性决定的。

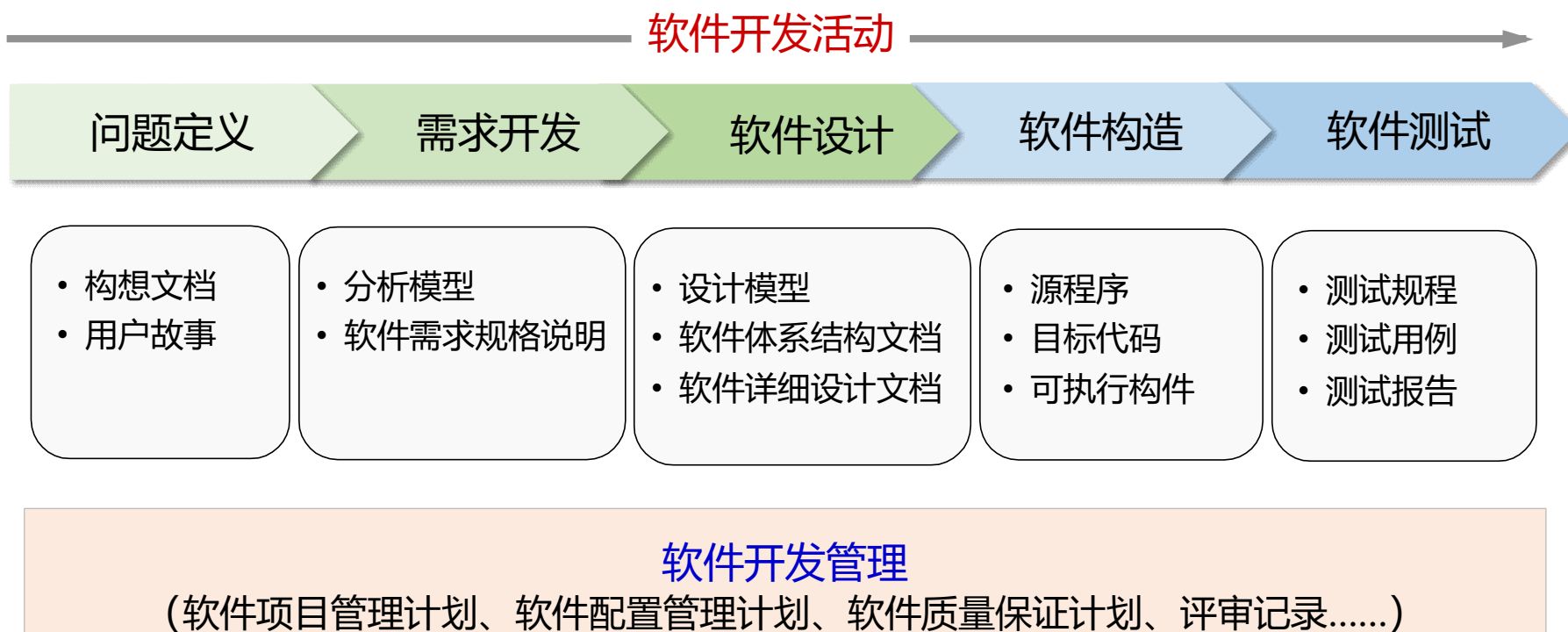
软件工程的基本要素



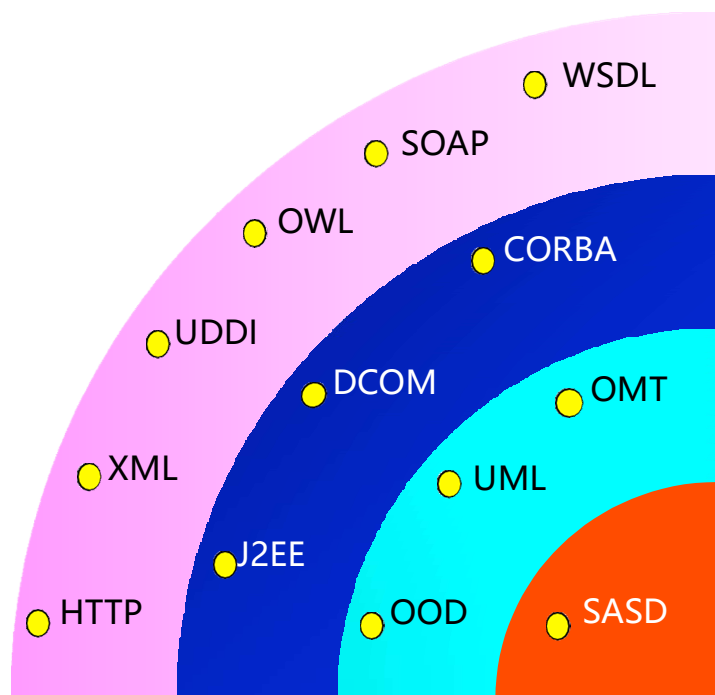
软件开发过程（一）



软件开发过程（二）



软件工程方法



- **面向服务**：在应用表现层次上将软件构件化，即应用业务过程由服务组成，而服务由构件组装而成。
- **面向构件**：寻求比类的粒度更大的且易于复用的构件，期望实现软件的再工程。
- **面向对象**：以类为基本程序单元，对象是类的实例化，对象之间以消息传递为基本手段。
- **面向过程**：以算法作为基本构造单元，强调自顶向下的功能分解，将功能和数据进行一定程度的分离。

软件工程工具

需求开发

软件设计

软件构造

软件测试

软件维护

开发管理

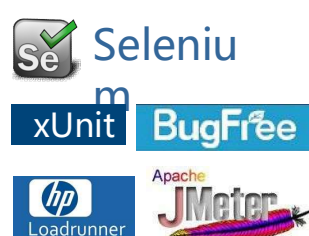
- 软件建模工具
- 数据库设计工具

- 程序编辑器
- 程序编译器
- 程序解释器
- 程序调试器
- 集成开发环境

- 单元测试工具
- 静态分析工具
- 自动化测试工具
- 性能测试工具
- 缺陷跟踪工具

- 代码重构工具
- 逆向工程工具

- 需求管理工具
- 项目管理工具
- 配置管理工具
- 测试管理工具



软件开发的基本策略

软件复用

- 构造一个新的系统不必从零做起，直接复用已有的构件进行组装
- 构件是经过反复使用验证的，由其组成的新系统具有较高的质量

分而治之

- 将一个复杂的问题分解成若干个简单的问题，然后逐个解决
- 来源于人们生活与工作的经验，完全适合于技术领域

逐步演进

- 软件开发是自底向上逐步有序的生长过程
- 小步快跑：每走完一步再调整并为下一步确定方向，直到终点

优化折中

- 优化：优化软件的各个质量特性，如运行速度、资源利用、用户体验
- 折中：通过协调各个质量特性，实现整体质量的最优

软件开发的基本策略：软件复用

软件复用是利用将已有的软件制品，直接组装或者合理修改形成新的软件系统，从而提高开发效率和产品质量，降低维护成本。

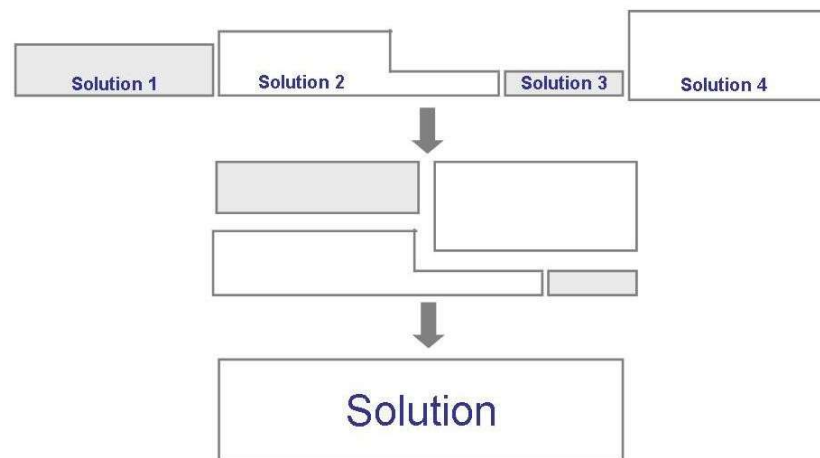
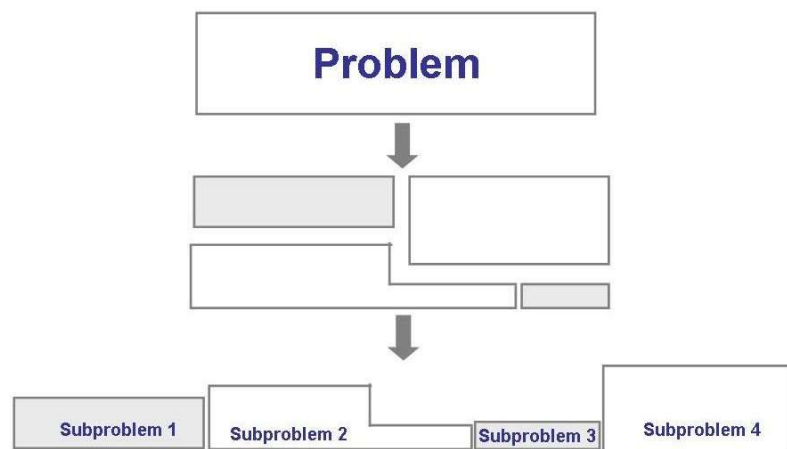
软件复用不仅仅是代码复用

- 库函数、类库
- 模板（文档、网页等）
- 设计模式
- 组件
- 框架



软件开发的基本策略：分而治之

软件工程是一项解决问题的工程活动，通过对问题进行研究分析，将一个复杂问题分解成可以理解并能够处理的若干小问题，然后再逐个解决。



软件开发的基本策略：逐步演进

软件更像一个活着的植物，其生长是一个逐步有序的过程。软件开发应该遵循软件的客观规律，不断进行迭代式增量开发，最终交付符合客户价值的产品。



软件开发的基本策略：优化折中

软件工程师应当把优化当成一种责任，不断改进和提升软件质量；但是优化是一个多目标的最优决策，在不可能使所有目标都得到优化时，需要进行折中实现整体最优。



在编写C程序代码时，对文件的访问是影响程序速度的一个重要因素，那么如何提高文件的访问速度呢？

软件开发的基本策略：优化折中

使用内存缓冲区方法，
读取1468802字节文件

缓冲大小	用户CPU (秒)	系统CPU (秒)	时钟时间 (秒)	循环次数 (秒)
1	23.8	397.9	423.4	1468802
2	12.3	202.0	215.2	734401
4	6.1	100.6	107.2	367201
8	3.0	50.7	54.0	183601
16	1.5	25.3	27.0	91801
32	0.7	12.8	13.7	45901
64	0.3	6.6	7.0	22951
128	0.2	3.3	3.6	11476
256	0.1	1.8	1.9	5738
512	0.0	1.0	1.1	2869
1024	0.0	0.6	0.6	1435
2048	0.0	0.4	0.4	718
4096	0.0	0.4	0.4	359
8192	0.0	0.3	0.3	180
16384	0.0	0.3	0.3	90
32768	0.0	0.3	0.3	45
65536	0.0	0.3	0.3	23
131072	0.0	0.3	0.3	12

对软件工程发展有重大影响的若干技术



个人软件过程

--Personal Software Process

• PSP0 • Planning • Development • Design • Coding • Test • Postmortem	• PSP0.1 • Planning • Development – Coding Standard – Design – Coding – Test • Size Measurement • Postmortem • Process Improvement Plan	• PSP1 • Planning – Estimate • Development – Coding Standard – Design – Coding – Test • Test Report • Size Measurement • Postmortem • Process Improvement Plan
---	--	---

PSP Evolution (cont.)

• PSP1.1 • Planning – Estimate • Development – Coding Standard – Design – Coding – Test • Record Time Spent • Test Report • Size Measurement • Postmortem • Process Improvement Plan	• PSP2 • Planning – Estimate • Development – Design Review – Coding Standard – Design – Coding – Code Review – Test • Record Time Spent • Test Report • Size Measurement • Postmortem • Process Improvement Plan	• PSP2.1 • Planning – Estimate • Development – Analysis – Design Spec – Design Review – Coding Standard – Design – Coding – Code Review – Test • Record Time Spent • Test Report • Size Measurement • Postmortem • Process Improvement Plan
---	---	--

Core Measures:

The PSP scripts provide expert-level guidance to following the process steps and they provide a framework for applying the PSP measures. The PSP has four core measures:

- **Size** – the size measure for a product part, such as lines of code (LOC).
 - How big is the task
- **Effort** – the time required to complete a task, usually recorded in minutes.
 - How big is the effort
- **Quality** – the number of defects in the product.
 - How good is your result
- **Schedule** – a measure of project progression, tracked against planned and actual completion dates.
 - How timely is the delivery

PSP ISSUES

- In small, start-up team, high quality requirement is hard to find
 - If the input is in low quality, how can the quality of PSP be ensured?
- Depends on data
 - Requires developers to enter data to log various activities
 - What if data is missing, or inaccurate?
 - Conflict of interest? Should I honestly record how bad I am?
- Micro-measurement
 - Do we need to measure code size everyday?
 - **Writing your own code vs. re-use, use libraries**
 - One strong dev removed 2000 LOC bad code, what's his performance?
- What's the end goal?
 - Measuring how to effectively execute on a requirement
 - NOT measuring how customers are satisfied by the product.

此题未设置答案，请点击右侧设置按钮

软件开发的基本要素不包括（）

提交

- ☐ A 开发过程
- ☐ B 开发人员
- ☐ C 开发方法
- ☐ D 开发工具

单选题 1分



此题未设置答案，请点击右侧设置按钮

软件开发的基本策略包括（）

提交

- ☐ A 逐步演进
- ☐ B 优化折中
- ☐ C 分而治之
- ☐ D 软件复用
- ☐ E 以上全都包括

此题未设置答案，请点击右侧设置按钮

处理软件复杂性的常用策略是（）。

提交

- ☐ A 逐步演进
- ☐ B 优化折中
- ☐ C 分而治之
- ☐ D 软件复用

本节内容

- 什么是软件工程：软件工程是从客户需求到编程实现之间的桥梁
- 什么是软件工程师：了解待开发软件的准确需求，规划、设计、开发和维护软件系统
- 什么是软件工程的目的是：经济有效地开发具有高质量的软件系统
- 什么是软件质量：质量模型
- 如何提高软件质量：软件工程的三要素过程、方法、工具
- 什么是个人软件流程（PSP）